

Plug-and-Play Linear Attention for Pre-trained Image and Video Restoration Models

Srinivasan Kidambi, and Pravin Nair, *Member, IEEE*

Abstract—Multi-head self-attention (MHSA) has become a core component in modern computer vision models. However, its quadratic complexity with respect to input length poses a significant computational bottleneck in real-time and resource-constrained environments. We propose PnP-Nystra, a Nyström-based linear approximation of self-attention, developed as a plug-and-play (PnP) module that can be integrated into the pre-trained image and video restoration models without retraining. As a drop-in replacement for MHSA, PnP-Nystra enables efficient acceleration in various window-based transformer architectures, including SwinIR, Uformer, and RVRT. Our experiments across diverse image and video restoration tasks, including denoising, deblurring, and super-resolution, demonstrate that PnP-Nystra achieves a 2–4× speed-up on an NVIDIA RTX 4090 GPU and a 2–5× speed-up on CPU inference. Despite these significant gains, the method incurs a maximum PSNR drop of only 1.5 dB across all evaluated tasks. To the best of our knowledge, we are the first to demonstrate a linear attention functioning as a training-free substitute for MHSA in restoration models.

Index Terms—Attention, Image and video restoration, Training-free, Plug-and-Play, Nyström approximation.

I. INTRODUCTION

Transformers have recently demonstrated strong performance across a range of low-level vision tasks, including image denoising, deblurring [1], super-resolution [2], and video restoration [3], [4]. These models rely on a self-attention mechanism that captures long-range dependencies by computing pairwise similarity between features corresponding to image or video patches. Let $\mathbf{X} \in \mathbb{R}^{N \times d_i}$ be the input feature map corresponding to N tokens (e.g., pixels or patches), where each token is represented by a d_i -dimensional feature vector. To compute self-attention [5], the input $\mathbf{X}$ is first projected into query, key, and value representations:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}_K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}_V,$$

where $\mathbf{W}_Q, \mathbf{W}_K \in \mathbb{R}^{d_i \times d}$, $\mathbf{W}_V \in \mathbb{R}^{d_i \times d_v}$ are learned projection matrices. Then, the attention matrix $\mathbf{A} \in \mathbb{R}^{N \times N}$, its row-normalized form $\mathbf{S} \in \mathbb{R}^{N \times N}$, and the self-attention output $\mathbf{O} \in \mathbb{R}^{N \times d_v}$ are computed as:

$$\mathbf{A} = \frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}}, \quad \mathbf{S} = \text{softmax}(\mathbf{A}), \quad \mathbf{O} = \mathbf{S}\mathbf{V}, \quad (1)$$

where $\text{softmax}(\mathbf{A})$ denotes independently applying softmax to each row of $\mathbf{A}$.

However, the attention operation in (1) incurs a quadratic complexity in the number of input tokens N , which corresponds to the number of pixels or patches within a subregion of

a frame [5]. In particular, the time and storage complexity for computing (1) is $\mathcal{O}(N^2d)$ and $\mathcal{O}(N^2 + Nd)$, respectively, dominated by storing and computing the large attention matrices $\mathbf{A}$ and $\mathbf{S}$. Currently, most vision models employ Multi-Head Self-Attention (MHSA) [5], an extension of self-attention. In MHSA, the embedding dimension d is divided into h attention heads, each of dimension d_h , such that $d = hd_h$. For each head, self-attention outputs are independently computed, concatenated, and linearly projected to form MHSA output. The time and storage complexity of MHSA is quadratic in N , similar to self-attention. This computational bottleneck makes transformers less suitable for real-time or resource-constrained settings, where convolutional networks are still often preferred due to their computational efficiency [6], [7].

Several approaches approximate self-attention via low-rank [8], [9], [10] or sparse [11] representations to reduce quadratic complexity, but these methods require retraining. Linformer [8] projects keys/values into lower-dimensional subspaces via learned linear maps, achieving linear complexity in N . Performer [9] replaces the softmax kernel with random feature approximation, resulting in linear complexity and unbiased attention estimates. Nyströmformer [10] heuristically reconstructs attention using landmark-based sampling. Axial Attention [12] factorizes 2D attention into 1D along spatial axes, reducing cost from H^2W^2 to $HW(H + W)$, where the number of tokens is $N = HW$. FlashAttention [13] and Triton [14] preserve exact attention while optimizing execution through custom fused GPU kernels but are hardware-specific and difficult to integrate into pre-trained models.

Interestingly, research on low-rank attention has seen reduced momentum since 2022, with the focus shifting to hardware-aware accelerations. In this work, we revisit classical low-rank approximations that can act as a *plug-and-play* replacement for attention modules in transformer-based vision models. Specifically, we propose **PnP-Nystra**, a training-free, plug-and-play approximation of self-attention using the Generalized Nyström approximation [15], [16], [17]. The core contributions of our method are summarized as follows:

- 1) Linear attention framework:** Unlike MHSA, which incurs quadratic time and memory complexity in the number of tokens N , PnP-Nystra achieves linear scaling in both time and memory.
- 2) Significant acceleration:** PnP-Nystra can replace standard attention layers in existing window-attention-based transformer models for image and video restoration, yielding substantial speed-ups. For example, in a video super-resolution setting with an upscaling factor of 2, we observe up to 4× speed-up on GPU compared to the original pre-trained model.
- 3) Robust performance without retraining:** Despite significant acceleration, the reduction in restoration quality is minimal.

The authors are with Department of Electrical Engineering, Indian Institute of Technology, Madras 600036, India (e-mail: ee21b139@smail.iitm.ac.in, pravin@iitm.ac.in).

For instance, in image denoising, PnP-Nystra achieves up to $3.5\times$ speed-up on CPU with < 1.3 dB PSNR reduction.

To the best of our knowledge, this is the first approach to enable inference-time replacement of self-attention in pre-trained transformers without any fine-tuning. This is especially valuable for real-time settings where retraining is impractical.

II. PROPOSED METHOD

In (1), by letting $\mathbf{G} = \exp(\mathbf{A})$ denote the element-wise exponential of the attention matrix $\mathbf{A}$, the self-attention output $\mathbf{O} \in \mathbb{R}^{N \times d_v}$ can be reformulated as

$$\mathbf{O} = (\mathbf{G}\mathbf{V}) \oslash (\mathbf{G}\mathbf{1}_N), \quad (2)$$

where $\mathbf{1}_N \in \mathbb{R}^N$ is the all-ones column vector, and $\oslash$ denotes row-wise element-wise division: that is, the i -th row of $\mathbf{G}\mathbf{V}$ is divided by the scalar $(\mathbf{G}\mathbf{1}_N)_i$. Thus, the attention output $\mathbf{O} \in \mathbb{R}^{N \times d_v}$ can be expressed in the kernel-based form from (2). Specifically, let the query matrix $\mathbf{Q} \in \mathbb{R}^{N \times d}$ and key matrix $\mathbf{K} \in \mathbb{R}^{N \times d}$ be viewed as stacks of row vectors $\{\mathbf{q}_i\}_{i=1}^N$ and $\{\mathbf{k}_j\}_{j=1}^N$, respectively. Then, the kernel matrix $\mathbf{G}$ in (2) is defined element-wise, using an exponential kernel, as

$$\mathbf{G}_{ij} = \exp\left(\frac{\mathbf{q}_i^\top \mathbf{k}_j}{\sqrt{d}}\right).$$

We propose to leverage the low-rank structure of the kernel matrix $\mathbf{G}$ via a generalized Nyström method adapted for non-symmetric matrices. Originally developed for approximating solutions to functional eigenvalue problems [18], [19], the Nyström method was later extended to efficiently estimate eigenvectors and construct low-rank matrix approximations [20], [21], [22]. To apply the Nyström approximation, we first select $m \ll N$ landmark query and key vectors, denoted by $\{\tilde{\mathbf{q}}_i\}_{i=1}^m$ and $\{\tilde{\mathbf{k}}_i\}_{i=1}^m$. Let $\tilde{\mathbf{Q}} \in \mathbb{R}^{m \times d}$ and $\tilde{\mathbf{K}} \in \mathbb{R}^{m \times d}$ denote the matrices formed by stacking these landmark vectors. We define the extended query and key matrices:

$$\tilde{\mathbf{Q}} = \begin{bmatrix} \tilde{\mathbf{Q}} \\ \mathbf{Q} \end{bmatrix}, \quad \tilde{\mathbf{K}} = \begin{bmatrix} \tilde{\mathbf{K}} \\ \mathbf{K} \end{bmatrix}.$$

We then define the extended kernel matrix $\tilde{\mathbf{G}} \in \mathbb{R}^{(m+N) \times (m+N)}$ as:

$$\tilde{\mathbf{G}} = \exp\left(\frac{\tilde{\mathbf{Q}}\tilde{\mathbf{K}}^\top}{\sqrt{d}}\right) = \begin{bmatrix} \mathbf{G}_A & \mathbf{G}_U \\ \mathbf{G}_L & \mathbf{G} \end{bmatrix}, \quad (3)$$

where, the core matrix $\mathbf{G}_A \in \mathbb{R}^{m \times m}$ captures similarities between landmark queries and keys:

$$\mathbf{G}_A = \exp\left(\frac{\tilde{\mathbf{Q}}\tilde{\mathbf{K}}^\top}{\sqrt{d}}\right). \quad (4)$$

The cross-similarity matrices are defined as $\mathbf{G}_L \in \mathbb{R}^{N \times m}$ (queries with landmark keys) and $\mathbf{G}_U \in \mathbb{R}^{m \times N}$ (landmark queries with keys):

$$\mathbf{G}_L = \exp\left(\frac{\mathbf{Q}\tilde{\mathbf{K}}^\top}{\sqrt{d}}\right), \quad \mathbf{G}_U = \exp\left(\frac{\tilde{\mathbf{Q}}\mathbf{K}^\top}{\sqrt{d}}\right). \quad (5)$$

Applying Nyström approximation to the extended matrix $\tilde{\mathbf{G}}$, the original kernel matrix $\mathbf{G}$ is approximated as:

$$\hat{\mathbf{G}} = \mathbf{G}_L \mathbf{G}_A^\dagger \mathbf{G}_U, \quad (6)$$

where $\mathbf{G}_A^\dagger$ denotes the Moore-Penrose pseudoinverse. Substituting (6) into (2), the attention output is approximated as:

$$\hat{\mathbf{O}} = (\hat{\mathbf{G}}\mathbf{V}) \oslash (\hat{\mathbf{G}}\mathbf{1}_N). \quad (7)$$

We refer to the approximation in (7) as **PnP-Nystra**, our method for approximating self-attention. The complete steps for computing PnP-Nystra are summarized in Algorithm 1.

Algorithm 1 PnP-Nystra in (7): Self-Attention Approximation

Require: Queries $\mathbf{Q} \in \mathbb{R}^{N \times d}$, Keys $\mathbf{K} \in \mathbb{R}^{N \times d}$, Values $\mathbf{V} \in \mathbb{R}^{N \times d_v}$, number of landmarks m

(Step 1) Select m landmark query and key vectors to form landmark matrices $\tilde{\mathbf{Q}} \in \mathbb{R}^{m \times d}$, $\tilde{\mathbf{K}} \in \mathbb{R}^{m \times d}$

(Step 2) Compute core matrix $\mathbf{G}_A \in \mathbb{R}^{m \times m}$ in (4)

(Step 3) Compute cross-similarity matrices $\mathbf{G}_L \in \mathbb{R}^{N \times m}$ and $\mathbf{G}_U \in \mathbb{R}^{m \times N}$ in (5)

(Step 4) Compute pseudoinverse $\mathbf{G}_A^\dagger \in \mathbb{R}^{m \times m}$

(Step 5) Compute premultiplier $\mathbf{P} \in \mathbb{R}^{N \times m}$ as $\mathbf{G}_L \mathbf{G}_A^\dagger$

(Step 6) Compute numerator $\mathbf{O}_N = \mathbf{P}(\mathbf{G}_U \mathbf{V}) \in \mathbb{R}^{N \times d_v}$ and compute denominator $\mathbf{O}_D = \mathbf{P}(\mathbf{G}_U \mathbf{1}_N) \in \mathbb{R}^{N \times 1}$

(Step 7) Normalize $\hat{\mathbf{O}} = \mathbf{O}_N \oslash \mathbf{O}_D$ (row-wise division)

return Output $\hat{\mathbf{O}} \in \mathbb{R}^{N \times d_v}$

Note that, while the Nyströmformer [10] also employs the Nyström method, it applies the Nyström method directly to the softmax-normalized attention matrix $\mathbf{S}$. Specifically, following the structure in Eq. 3, the authors define:

$$\tilde{\mathbf{S}} = \text{softmax}\left(\frac{\tilde{\mathbf{Q}}\tilde{\mathbf{K}}^\top}{\sqrt{d}}\right).$$

Unlike the kernel matrix $\tilde{\mathbf{G}}$ in our formulation, $\tilde{\mathbf{S}}$ cannot be block-partitioned due to row-wise normalization, which induces global coupling among elements. Consequently, a direct Nyström factorization is not applicable. Instead, [10] proposes a heuristic approximation:

$$\mathbf{S} \approx \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}}\right) \left[\text{softmax}\left(\frac{\tilde{\mathbf{Q}}\tilde{\mathbf{K}}^\top}{\sqrt{d}}\right) \right]^\dagger \text{softmax}\left(\frac{\tilde{\mathbf{Q}}\mathbf{K}^\top}{\sqrt{d}}\right)$$

While computationally efficient, this construction lacks theoretical approximation guarantees such as those in Lemma 2.1, in contrast to our PnP-Nystra, which is grounded in provable low-rank kernel approximations.

A. Computational Details

Standard self-attention incurs a time complexity of $\mathcal{O}(N^2 d)$ and memory usage of $\mathcal{O}(N^2 + Nd)$. In contrast, our proposed method reduces both significantly by avoiding full $N \times N$ attention matrix computation. The per-step complexity is summarized in Table I. The overall time complexity is reduced to $\mathcal{O}(Nm(d + d_v))$ and memory usage is reduced to $\mathcal{O}(m^2 + N(m + d_v))$ which is significantly lower than standard attention when $m \ll N$.

In Step 1 of Algorithm 1, we construct landmark queries and keys by averaging feature tokens over non-overlapping spatial windows [23]. This approach is computationally efficient and well-aligned with the local image statistics. In Step 4,

TABLE I: Per-Step Time and Space Complexity of PnP-Nystra

Complexity	Step 2	Step 3	Step 4	Step 5	Step 6	Step 7
Time	$\mathcal{O}(m^2d)$	$\mathcal{O}(Nmd)$	$\mathcal{O}(m^3)$	$\mathcal{O}(Nm^2)$	$\mathcal{O}(Nm(d_v + 1))$	$\mathcal{O}(Nd_v)$
Space	$\mathcal{O}(m^2)$	$\mathcal{O}(Nm)$	$\mathcal{O}(m^2)$	$\mathcal{O}(Nm)$	$\mathcal{O}(N(d_v + 1))$	in-place

we observed that the built-in PyTorch pseudoinverse function produces numerically unstable results in practice. To address this, we adopt the robust pseudoinverse computation technique from [24], an iterative algorithm. Additionally, to avoid overflow during kernel computation in Steps 2 and 3, we replace large exponentials beyond a threshold with linear approximations to ensure numerical stability.

B. Approximation error

We analyze the spectral norm error of the Nyström-based kernel approximation used in PnP-Nystra, where the original matrix $\mathbf{G}$ is approximated as $\hat{\mathbf{G}} = \mathbf{G}_L \mathbf{G}_A^\dagger \mathbf{G}_U$.

Lemma 2.1 (Spectral norm error bound): Assume $\mathbf{G}_A$ is nonsingular. Then,

$$\|\mathbf{G} - \mathbf{G}_L \mathbf{G}_A^\dagger \mathbf{G}_U\|_2 \leq C \frac{\sigma_{m+1}(\tilde{\mathbf{G}})}{\sigma_m(\mathbf{G}_A)},$$

where $\sigma_k(\cdot)$ denotes the k -th singular value, and the constant C depends quadratically on $\sigma_1(\mathbf{G})$.

The proof follows from Lemma 5.10 in [15], along with the fact that the spectral norm error between $\mathbf{G}$ and $\hat{\mathbf{G}}$ is upper bounded by that between $\tilde{\mathbf{G}}$ and its Nyström approximation ($\hat{\mathbf{G}}$ instead of $\mathbf{G}$ in (3)). The error bound in Lemma 2.1 indicates that the approximation error decreases when the singular values of $\tilde{\mathbf{G}}$ decay rapidly. In particular, if the number of landmark points m approaches the rank of $\tilde{\mathbf{G}}$, then $\sigma_{m+1}(\tilde{\mathbf{G}}) \rightarrow 0$, leading to tighter approximations. The following result formalizes this by guaranteeing perfect reconstruction when the number of landmark points equals the rank of the extended kernel matrix.

Corollary 2.2 (Exact recovery): If $\text{rank}(\tilde{\mathbf{G}}) = m$, then $\sigma_{m+1}(\tilde{\mathbf{G}}) = 0$, and the Nyström approximation becomes exact, i.e., $\mathbf{G} = \mathbf{G}_L \mathbf{G}_A^\dagger \mathbf{G}_U$.

We next empirically validate the low-rank property of the kernel matrix in pre-trained vision models, which is indeed the assumption underlying Lemma (2.1) and Corollary (2.2). Fig. 1 illustrates the singular value spectrum of the attention matrices from two distinct pre-trained models. In both cases, the rapid singular value decay confirms the low-rank structure of the attention matrices.

III. EXPERIMENTS

We evaluate drop-in replacement capabilities of PnP-Nystra in existing restoration models, followed by an ablation study of its algorithmic parameters. PSNR and SSIM metrics for the experiments are reported, alongside runtime and speed-up on both GPU and CPU platforms. All timings are averaged over multiple inference runs on a single input sample per test dataset. Experiments are conducted using PyTorch 2.7 with CUDA 12.2, with GPU and CPU benchmarks performed on NVIDIA RTX 4090 and Intel Xeon Gold processors, respectively. The implementation is available at: https://github.com/Srinivas-512/PnP_Nystra.

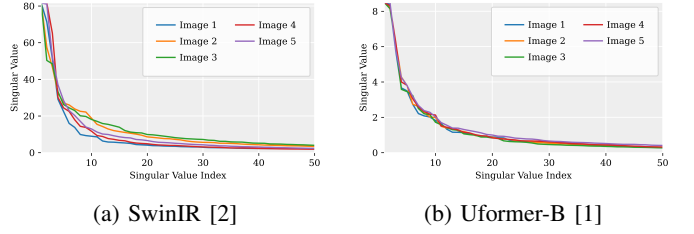


Fig. 1: Variation of the top 50 singular values of attention maps ($N = 32^2$) from SwinIR and Uformer, averaged over all heads and layers. The steep decay within the first 20 singular values highlights the low-rank structure of the attention matrices.

A. PnP-Nystra for restoration

In this section, we evaluate PnP-Nystra as a drop-in replacement for self-attention layers in three state-of-the-art restoration models: SwinIR [2], Uformer-B [1], and RVRT [4]. All projection weights ($\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V$) are retained (no fine-tuning performed) in the pre-trained models. Landmark count $m \in \{16, 32\}$ and pseudoinverse iterations (2-6) are selected to optimize performance across all experiments. We emphasize that our evaluations are restricted to the original pre-trained models, which already deliver state-of-the-art restoration performance. Our goal is not exhaustive benchmarking for restoration but to assess drop-in approximation within high-performing baselines.

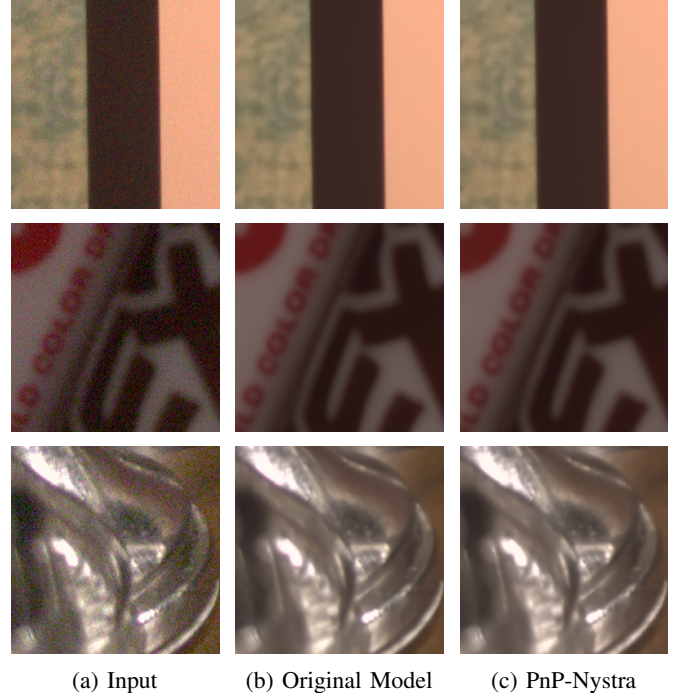


Fig. 2: Comparison of three image patches from Uformer-B and its PnP-Nystra variant for image denoising.

1) *Image denoising:* Table II presents results for Uformer-B with PnP-Nystra on SIDD and BSDS200 denoising benchmarks, where the window size is 64, i.e., number of tokens $N = 64^2$. Across two input resolutions, our method achieves $2\text{--}2.7\times$ GPU and $3.6\text{--}5.2\times$ CPU speed-up, with PSNR drop under



Fig. 3: $2\times$ super-resolution with SwinIR: PnP-Nystra closely matches the original pre-trained model in PSNR (in dB), SSIM, and visual quality. Error maps (scaled to $[0-255]$) show pixel-level differences from ground-truth, confirming visual fidelity.

1.3 dB and SSIM loss below 0.04. Also shown in Fig. 2 is an additional representative result, highlighting that PnP-Nystra maintains visual fidelity comparable to the original pre-trained model across diverse image patches and structures. These results demonstrate significant efficiency gains with minimal degradation.

TABLE II: Comparison of Uformer-B and Its PnP-Nystra Variant for Image Denoising

Timing Analysis (GPU / CPU, in ms)			
Resolution	Original	PnP-Nystra	Speedup
512×512	219.33 / 5760	81.96 / 1580	$2.67\times$ / $3.65\times$
256×256	88.77 / 2310	42.41 / 440	$2.09\times$ / $5.25\times$
Accuracy Analysis (PSNR in dB / SSIM)			
Dataset	Original	PnP-Nystra	Drop
SIDD	38.89 / 0.8950	37.73 / 0.8836	1.16 / 0.0114
BSDS200	28.05 / 0.8067	26.81 / 0.7748	1.24 / 0.0319

TABLE III: Comparison of SwinIR and Its PnP-Nystra Variant for Image Super-resolution for different scale factors

Timing Analysis (GPU / CPU in ms)				
Resolution	Scale	Original	PnP-Nystra	Speedup
288×288	2	557.32 / 15330	240.80 / 6902	$2.31\times$ / $2.22\times$
160×160	4	165.33 / 4380	62.64 / 1460	$2.64\times$ / $3.00\times$
96×96	8	60.02 / 1810	28.28 / 570	$2.12\times$ / $3.17\times$
Accuracy Analysis (PSNR in dB / SSIM)				
Dataset	Scale	Original	PnP-Nystra	Drop
Set5	2	36.05 / 0.9450	35.30 / 0.9412	0.75 / 0.0038
	4	30.71 / 0.8701	29.13 / 0.8449	1.58 / 0.0252
	8	25.60 / 0.7403	24.40 / 0.6868	1.20 / 0.0535
BSDS100	2	30.98 / 0.8922	30.27 / 0.8830	0.71 / 0.0092
	4	26.43 / 0.7183	25.67 / 0.6946	0.76 / 0.0237
	8	23.61 / 0.5680	23.04 / 0.5441	0.57 / 0.0239

2) *Image super-resolution*: Table III compares SwinIR and its PnP-Nystra variant on Set5 and BSDS100 across different scales for a window size of 32 i.e. $N = 32^2$. PnP-Nystra consistently achieves $2 - 3\times$ speed-up on both GPU and CPU, with better runtime benefits at lower resolutions. The accuracy drop, measured in PSNR and SSIM, is low considering the speed-up obtained. We also demonstrate in Fig. 3 that PnP-Nystra achieves super-resolution results visually on par with the original SwinIR, with negligible perceptual difference.

3) *Image deblurring*: Table IV reports the performance of PnP-Nystra on the RealBlur R dataset using Uformer-B for image deblurring, where window-size is 32, i.e., $N = 32^2$.

Our method achieves a GPU speed-up of $1.76\times$ and a CPU speed-up of $1.87\times$ compared to the original model, with no degradation in performance (negligible change in PSNR and SSIM values). This shows that there exist cases where PnP-Nystra provides near-to-exact reconstruction as the original pre-trained models.

TABLE IV: Comparison of Uformer-B and Its PnP-Nystra Variant for Image Deblurring

Timing Analysis (GPU/ CPU, in ms)			
Resolution	Original	PnP-Nystra	Speedup
768×768	305.16 / 7920	173.41 / 4220	$1.76\times$ / $1.87\times$
Accuracy Analysis (PSNR / SSIM)			
Dataset	Original	PnP-Nystra	Accuracy Drop
RealBlur R	33.98 / 0.9404	34.00 / 0.9404	-0.02 dB / 0.0000

4) *Video super-resolution*: Table V shows the runtime and accuracy of PnP-Nystra when integrated into RVRT for $4\times$ video super-resolution, where N is 64^2 , i.e., the window size is 64. Compared to the original model, our method achieves over $4\times$ inference time reduction on REDS4 and Vid4 datasets, with minimal degradation of 1.3-1.9 dB in PSNR and 0.04-0.05 in SSIM. Thus PnP-Nystra is an effective accelerating strategy for high-resolution video restoration. Due to hardware-specific compilation issues, we could not execute RVRT and its PnP-Nystra variant in the CPU.

TABLE V: Comparison of RVRT and its PnP-Nystra variant for $\times 4$ Video Super-Resolution

Timing Analysis (GPU, in ms)				Accuracy Analysis (PSNR / SSIM)		
Dataset	Original	PnP-Nystra	Speedup	Original	PnP-Nystra	Drop
REDS4	849.53	207.77	4.09	30.83 / 0.8745	28.95 / 0.8338	1.88 / 0.0407
Vid4	850.43	208.44	4.08	26.22 / 0.8292	24.84 / 0.7835	1.38 / 0.0457

B. Ablation Study of PnP-Nystra

In this section, we present an ablation study of PnP-Nystra, with emphasis on the internal hyper-parameters.

Approximation Error: We compute the approximation error between the attention matrix $\mathbf{S}$ of the original model and its PnP-Nystra variant, averaged over heads and layers. Note that $\mathbf{S}$ is not explicitly computed during our inference. The average mean square error between the normalized attention matrix $\mathbf{S}$ from the original model and PnP-Nystra are recorded in Table VI. Note that elements in the matrix are in the range 0 to 1.

TABLE VI: Attention Matrix MSE Errors for Window Attention Approximation via PnP-Nystra

Application	Dataset	Scale	MSE Error
SwinIR image super-resolution	Set5	$\times 2$	1.19×10^{-5}
		$\times 4$	1.05×10^{-5}
		$\times 8$	1.24×10^{-5}
	BSDS100	$\times 2$	1.14×10^{-5}
		$\times 4$	1.04×10^{-5}
		$\times 8$	1.32×10^{-5}
Uformer image denoising	SIDD		3.78×10^{-5}
	BSDS200		2.00×10^{-5}
Uformer image deblurring	RealBlur R		6.11×10^{-6}
RVRT video super-resolution	REDS4		3.89×10^{-5}
	Vid4		3.00×10^{-3}

PnP-Nystra achieves consistently low approximation errors in the range 10^{-5} - 10^{-6} , across various restoration models.

Attention map comparison: Since PnP-Nystra approximates the kernel matrix $\mathbf{G}$, it implicitly approximates the attention map $\mathbf{S}$. Fig. 4 shows that PnP-Nystra yields attention patterns closely matching those of the original models.

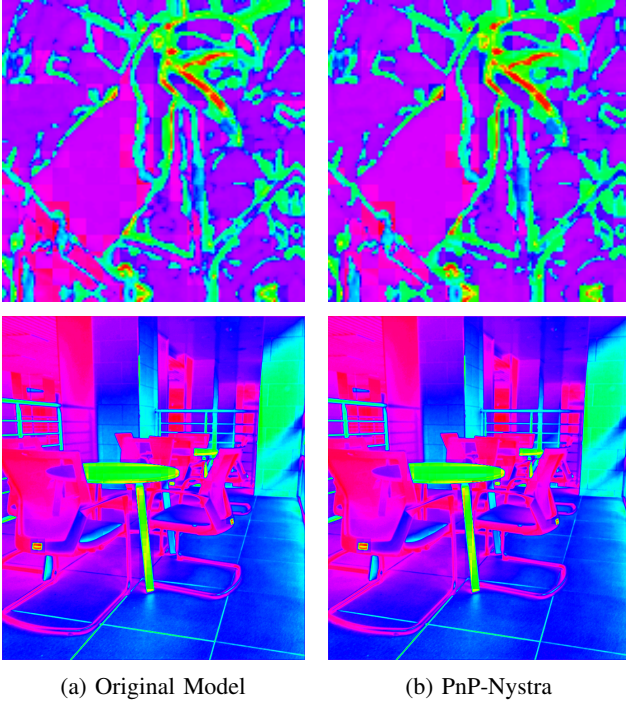
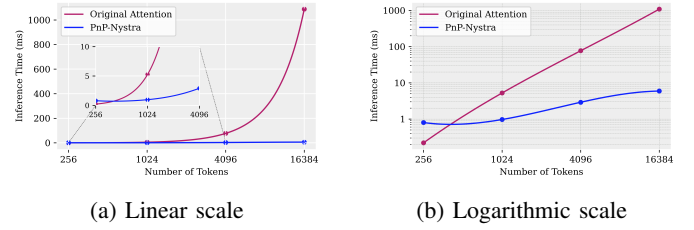


Fig. 4: Comparison of attention maps for original model and PnP-Nystra for SwinIR (top) and Uformer-B (bottom).

Scaling of Inference Time with N : Fig. 5 presents the CPU inference runtime of the original MHSA module versus PnP-Nystra (averaged over repeated runs), plotted in both linear and logarithmic scales with respect to the token count N . In line with the complexity analysis, PnP-Nystra exhibits linear scaling with an increasing number of tokens.

Landmarks and Pseudoinverse iterations: We evaluate the effect of varying the number of landmarks m and the number of iterations used for pseudoinverse computation in Step 4 of Algorithm 1. Table VII reports PSNR and SSIM drop

Fig. 5: Inference time vs. token count N : Unlike MHSA which grows quadratically with N , PnP-Nystra scales linearly.

for the PnP-Nystra variant of Uformer-B for denoising on the SIDD dataset. As expected from Nyström approximation, the approximation performance tends to the original with an increase in number of landmarks. Additionally, a better estimate of the pseudoinverse leads to improved approximation.

TABLE VII: Impact of Number of Landmarks and Pseudoinverse Iterations on Approximation Accuracy

Varying # Landmarks (at 6 Iterations)			Varying Iterations (at 32 Landmarks)		
# Landmarks	PSNR Drop	SSIM Drop	Iterations	PSNR Drop	SSIM Drop
8	1.43	0.0127	1	2.38	0.0164
16	1.23	0.0119	3	1.57	0.0137
32	1.16	0.0114	6	1.16	0.0114

IV. CONCLUSION AND FUTURE WORK

We presented PnP-Nystra, a plug-and-play Nyström approximation for self-attention that enables efficient inference in window-based transformer models for image and video restoration. Experiments show that PnP-Nystra acts as a fast, training-free replacement for self-attention in pre-trained models, offering a viable solution for resource-constrained deployment. Future work includes improving the speed-up of PnP-Nystra and extending the framework to global attention in vision transformers and diffusion transformers.

REFERENCES

- [1] Z. Wang, X. Cun, J. Bao, W. Zhou, J. Liu, and H. Li, "Uformer: A general u-shaped transformer for image restoration," *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 17 683–17 693, 2022.
- [2] J. Liang, J. Cao, G. Sun, K. Zhang, L. Van Gool, and R. Timofte, "Swinir: Image restoration using swin transformer," *Proc. IEEE/CVF International Conference on Computer Vision*, pp. 1833–1844, 2021.
- [3] J. Liang, J. Cao, Y. Fan, K. Zhang, R. Ranjan, Y. Li, R. Timofte, and L. Van Gool, "Vrt: A video restoration transformer," *IEEE Transactions on Image Processing*, vol. 33, pp. 2171–2182, 2024.
- [4] J. Liang, Y. Fan, X. Xiang, R. Ranjan, E. Ilg, S. Green, J. Cao, K. Zhang, R. Timofte, and L. V. Gool, "Recurrent video restoration transformer with guided deformable attention," *Proc. Advances in Neural Information Processing Systems*, vol. 35, pp. 378–393, 2022.
- [5] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Proc. Advances in neural information processing systems*, vol. 30, 2017.
- [6] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, "A convnet for the 2020s," *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 11 976–11 986, 2022.
- [7] B. Ren, Y. Li, N. Mehta, R. Timofte, H. Yu, C. Wan, Y. Hong, B. Han, Z. Wu, Y. Zou *et al.*, "The ninth NTIRE 2024 efficient super-resolution challenge report," *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 6595–6631, 2024.
- [8] S. Wang, B. Z. Li, M. Khabsa, H. Fang, and H. Ma, "Linformer: Self-attention with linear complexity," *arXiv preprint arXiv:2006.04768*, 2020.

- [9] K. Choromanski, V. Likhoshesterov, D. Dohan, X. Song, A. Gane, T. Sarlos, P. Hawkins, J. Davis, A. Mohiuddin, L. Kaiser *et al.*, “Rethinking attention with performers,” *Proc. International Conference on Learning Representations*, 2021.
- [10] Y. Xiong, Z. Zeng, R. Chakraborty, M. Tan, G. Fung, Y. Li, and V. Singh, “Nyströmformer: A nyström-based algorithm for approximating self-attention,” *Proc. AAAI Conference on Artificial Intelligence*, vol. 35, no. 16, pp. 14 138–14 148, 2021.
- [11] Z. Gao, Z. Tong, L. Wang, and M. Z. Shou, “Sparseformer: Sparse visual recognition via limited latent tokens,” *Proc. International Conference on Learning Representations*, 2024.
- [12] J. Ho, N. Kalchbrenner, D. Weissenborn, and T. Salimans, “Axial attention in multidimensional transformers,” *arXiv preprint arXiv:1912.12180*, 2019.
- [13] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré, “Flashattention: Fast and memory-efficient exact attention with io-awareness,” *Proc. Advances in neural information processing systems*, vol. 35, pp. 16 344–16 359, 2022.
- [14] P. Tillet, H.-T. Kung, and D. Cox, “Triton: An intermediate language and compiler for tiled neural network computations,” *Proc. ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, pp. 10–19, 2019.
- [15] A. A. Nemtsov and A. Schclar, “Matrix compression using the Nyström method,” *Intelligent Data Analysis*, vol. 20, no. 5, pp. 997–1019, 2016.
- [16] M. W. Mahoney and P. Drineas, “CUR matrix decompositions for improved data analysis,” *Proc. National Academy of Sciences*, vol. 106, no. 3, pp. 697–702, 2009.
- [17] S. Wang and Z. Zhang, “Improving CUR matrix decomposition and the Nyström approximation via adaptive sampling,” *The Journal of Machine Learning Research*, vol. 14, no. 1, pp. 2729–2769, 2013.
- [18] E. J. Nyström, “Über die praktische auflösung von integralgleichungen mit anwendungen auf randwertaufgaben,” *Acta Mathematica*, vol. 54, no. 1, pp. 185–204, 1930.
- [19] C. T. Baker, *The Numerical Treatment of Integral Equations*. Clarendon Press, 1977.
- [20] H. Talebi and P. Milanfar, “Global image denoising,” *IEEE Transactions on Image Processing*, vol. 23, no. 2, pp. 755–768, 2014.
- [21] C. Williams and M. Seeger, “Using the Nyström method to speed up kernel machines,” *Proc. Advances in neural information processing systems*, vol. 13, 2000.
- [22] C. Fowlkes, S. Belongie, F. Chung, and J. Malik, “Spectral grouping using the Nyström method,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 26, no. 2, pp. 214–225, 2004.
- [23] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo, “Swin transformer: Hierarchical vision transformer using shifted windows,” *Proc. IEEE/CVF International Conference on Computer Vision*, pp. 10 012–10 022, 2021.
- [24] M. K. Razavi, A. Kerayechian, M. Gachpazan, and S. Shateyi, “A new iterative method for finding approximate inverses of complex matrices,” *Abstract and Applied Analysis*, vol. 2014, no. 1, p. 563787, 2014.